



실습 중심 강의

Coding Agent 기본

AI를 활용한 SW 개발 기초

PRD → GitHub → 배포 · 프롬프트 · 컨텍스트 · 하네스 엔지니어링을 직접 따라 하는 실습

실습 프로젝트

Tetris — 프롬프트 복붙으로 브라우저 게임을 만들어 GitHub Pages에 배포

개념부터 배포까지 — 프롬프트를 복붙해 브라우저 게임을 만들고 인터넷에 무료로 공개합니다.

과정 개요 — 프롬프트 복붙으로 PRD에서 배포까지

항목	내용
교육 대상	프로그래밍 경험이 없어도 프롬프트 복붙으로 완주 — 누구나 참여
교육 시간	사전 준비(Part 1, 20-30분) + 이론(Part 2-6) → 실습(Part 7-8) → 회고(Part 9)
교육 방식	Codex App(데스크톱/웹)을 메인으로 '직접 손으로' 실습, CLI는 보조
도구 스택	순수 HTML/CSS/JavaScript · Node.js · Git · Codex App (설치할 것이 거의 없음)
사전 과제	계정: ChatGPT·OpenAI · GitHub / 설치: Node.js 18+ · Git
심화 옵션	게임 다듬기(선택) — 다음 블록 미리보기 · 레벨 · 속도 증가
관통 예제	Tetris — 프롬프트 복붙으로 만드는 브라우저 게임 (GitHub Pages 배포)

핵심 목표 — 프롬프트·컨텍스트·하네스로 PRD → GitHub → 배포까지, 내 손으로 제품을 출시·운영한다.

학습 목표 — 과정을 마치면 할 수 있는 것

1

전체 기술 지도를 설명

Codex의 사용 방식·실행 환경·지시/권한 체계 등 전체 기술 지도를 설명할 수 있다.

2

규칙·검증 체계 구성

Prompt·Context·Harness와 AGENTS.md로 프로젝트 규칙과 검증 체계를 구성할 수 있다.

3

안전한 실행 설정

Sandbox·Approvals로 안전하게 실행하고 config.toml 기초를 설정할 수 있다.

4

제품 출시·운영

PRD → GitHub → 배포까지 제품을 출시·운영할 수 있다.



PART 1

시작 전 준비

계정과 환경

실습 중 '계정이 없어서' 막히지 않도록, 본격적인 실습 전에 계정과 도구를 먼저 준비합니다. 약 20-30분 정도 걸립니다

🕒 1교시 · 이론

1.1 시작 전 준비 — 계정과 환경

공통

실습 중 막히지 않도록 계정·도구를 먼저 준비 (약 20-30분)

만들어 둘 계정 (모두 무료로 시작)

ChatGPT / OpenAI

Codex 로그인·모델 사용 — chatgpt.com

GitHub

소스 코드 저장·배포 트리거 — github.com

로컬 개발 도구 설치

필요한 것은 Node.js와 Git 뿐입니다.

설치 여부 확인

```
node -v
```

```
git --version
```

Node.js 18 이상 (20 LTS 권장)

사전 점검 체크리스트

✅ ChatGPT/OpenAI 로그인 가능

✅ GitHub 로그인됨

✅ node -v 버전 출력

✅ git --version 버전 출력



PART 2

Codex 개관

사용 방식(App·IDE·CLI)과 Codex App 설치

Codex가 무엇인지 전체 기술 지도를 개관하고, App·IDE·CLI 사용 방식과 Codex App 설치·첫 작업까지 살펴봅니다.

🕒 1교시 · 이론

2.1 Codex 전체 기술 지도 — 네 개의 축

공통

Codex는 터미널·에디터에서 실제로 코드를 읽고 고치고 실행하는 코딩 에이전트

1 모델

코드를 생성·추론하는 AI 엔진 (gpt-5.x-codex 계열)

다루는 곳 **Part 2 · 6.5**

2 사용 방식

Codex를 '어디서' 쓰는가 — App · IDE 확장 · CLI

다루는 곳 **Part 2**

3 지시·권한

AGENTS.md(규칙) · Sandbox(범위) · Approvals(승인)

다루는 곳 **Part 5 · 6**

4 하네스

빌드·테스트·배포·로그로 결과를 '검증'하는 틀

다루는 곳 **Part 6**

이 네 축이 결과 품질을 좌우합니다: 무엇을 시키는가(프롬프트) · 무엇을 보여주는가(컨텍스트) · 어떤 검증 틀에서 일하게 하는가(하네스).

2.2 핵심 용어 미리 보기

지금 다 외울 필요는 없습니다 — 각 용어는 해당 파트에서 실습으로 익힙니다

Prompt

Codex에게 시키는 자연어 지시. 작게 쪼개고 검증 방법을 함께 준다

Context

Codex가 참고하는 정보(파일·이미지·대화·AGENTS.md). 모델의 '작업 기억'

AGENTS.md

프로젝트 상시 규칙 문서. 모든 작업 전에 Codex가 먼저 읽는다

Harness

작업을 감싸는 검증·안전 장치 (빌드/테스트/린트/샌드박스)

Sandbox

Codex가 파일·네트워크를 쓸 수 있는 '기술적 범위'

Approvals

위험한 행동 전에 사람에게 물어보게 하는 '정책'

Thread

하나의 작업 세션. 여러 프롬프트가 이어진 대화

MCP

Codex에 외부 도구(GitHub 등)를 연결하는 표준 규약

2.3 사용 방식 — App · IDE · CLI

어떤 방식으로 쓰든 실제로 일하는 AI 모델은 똑같습니다

App

MAIN

독립 데스크톱/웹 앱. 인앱 브라우저·변경 미리보기·승인 UI·클라우드 작업

언제 쓰나 이 과정의 메인 실습

IDE 확장

VS Code 등 에디터에 통합. 열린 파일·선택 영역을 자동 컨텍스트화

언제 쓰나 코드 작성 중 즉시 도움

CLI

터미널 기반. 스크립트/CI 자동화에 적합 (Rust로 작성되어 가볍고 빠름)

언제 쓰나 자동화·보조

[NOTE] 왜 App을 메인으로? App은 Codex가 제안한 파일 변경을 diff(전/후 비교)로 보여주고, 승인·미리보기·클라우드 작업을 한 화면에서 다룰 수 있어 입문자에게 가장 안전합니다.

2.4 ▶ 실습 — App 설치부터 첫 작업까지

설치 → 폴더 열기 → 첫 지시 → 결과 확인

1 설치 & 로그인

openai.com에서 Codex 데스크톱 앱 설치 → 같은 ChatGPT 계정으로 로그인.
기본값은 '작업 폴더 안에서만 변경, 위험한 동작은 승인'

2 첫 프로젝트 열기

빈 폴더(예: codex-hello)를 만들고 App에서 '폴더 열기'. 이 폴더가 Codex의
작업 공간이 된다

3 첫 작업 — 인사하기

Codex App 작성 칸

폴더에 hello.txt 파일을 만들고
"Hello Codex" 한 줄을 써줘.

→ 파일 목록에서 hello.txt 를 열어
내용이 "Hello Codex"인지 확인

이어지는 실습: 다양한 일 시켜 보기

· 엑셀 VLOOKUP 예시 만들기

· 다운로드 폴더 정리 방법

· CSV 파일이 어떤 데이터인지 설명

· 이미지 5장 생성해 저장

2.5 Codex App 사이드바 핵심 기능

웹 만들어 공유 · 반복 작업 자동화 · 기능 확장



사이트 (Sites)

프롬프트 한 줄로 웹페이지·앱·게임을 만들어 OpenAI 서버에 바로 호스팅. 미리보기
→ 공개 범위 선택 → 링크 공유

Codex에게 맡기는 배포



예약 (Scheduled)

'나중에·정기적으로' 할 일을 맡기기. 매일/매주 반복, 변화 감시. 실행 결과는 예약 화면에 수신함처럼 모임

반복 작업 자동화



플러그인 (Plugins)

새 기능(스킬·앱·템플릿)을 더하는 확장 장터. 문서·PDF·스프레드시트·프레젠테이션이 기본 포함

기능 확장

[TIP] 세 기능은 서로 연결됩니다 — 예약 작업이 플러그인·스킬을 함께 호출할 수 있습니다.



PART 3

프롬프트 엔지니어링

받은 만큼 일한다 — 정의 · 26가지 원칙 · P·C·T·F

프롬프트는 Codex에게 주는 '작업 지시문'입니다. 핵심은 (1) 작업을 작고 명확하게 쪼개고, (2) '끝났는지 아는 법(검증)'을 함께 주는 것입니다.

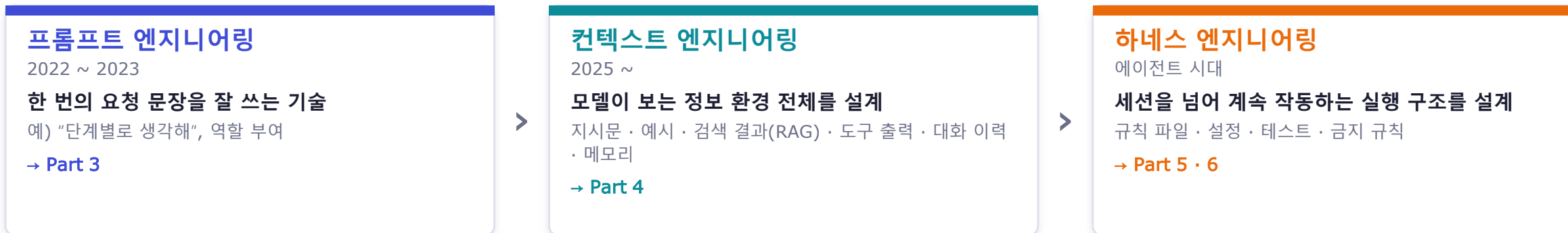
🕒 1교시 · 이론

3.1 프롬프트 엔지니어링의 정의

같은 기술을 사람마다 다르게 정의합니다 — 공통점은 '자연어로 모델을 프로그래밍한다'는 것

인물 · 시점	정의 · 발언	관점
안드레이 카파시 2023.1 · X 게시물	"가장 인기 있는 새 프로그래밍 언어는 영어다." <i>The hottest new programming language is English</i>	자연어가 곧 코드
안드레이 카파시 2025.6 · 강연 「Software 3.0」	소프트웨어 1.0은 코드, 2.0은 신경망 가중치, 3.0은 프롬프트가 곧 프로그램. LLM은 새로운 운영체제에 가깝다.	프롬프트 = 프로그램
앤드류 응 2023 · 「ChatGPT Prompt Engineering for Developers」	① 명확하고 구체적인 지시를 써라 ② 모델에게 생각할 시간을 줘라 + 한 번에 완벽한 프롬프트는 없다 — 반복적으로 개선하라	정의보다 실천 원칙

정의의 변화 흐름 — 한 줄 지시에서 실행 구조로



출처 · A. Karpathy, X(2023.1.24) · A. Karpathy, 「Software Is Changing (Again)」 YC AI Startup School(2025.6) · A. Ng & I. Fulford, DeepLearning.AI 「ChatGPT Prompt Engineering for Developers」(2023)

3.2 프롬프트 엔지니어링 — 나쁨 vs 좋음

Codex는 받은 만큼 일합니다

× 나쁜 프롬프트

로그인 기능 만들어줘.

대상 파일 · 동작 규칙 · 검증 방법이 모두 빠짐
→ Codex가 알아서 추측 → 결과가 매번 달라진다

✓ 좋은 프롬프트

src/auth.ts 에 이메일+비밀번호 로그인을 구현하고,
실패 시 에러 반환. 완료 후 npm test로 검증해줘.

① 대상 파일 지정 ② 동작 규칙 ③ 반환 형태 ④ 검증 방법
→ 결과가 예측 가능하고, Codex가 스스로 검증까지

좋은 프롬프트의 두 가지 핵심 원칙

→ 이 둘을 매번 챙기는 체크리스트가 P · C · T · F

1 작업을 작고 명확하게 쪼갬다

큰 작업은 '먼저 계획을 세우라'고 시킨 뒤 한 단계씩 진행

2 '끝났는지 어떻게 아는지'를 함께 준다

테스트·실행·린트 등 스스로 검증할 방법을 포함

3.3 재사용 템플릿 — P · C · T · F

네 글자만 외우면 거의 모든 작업에 적용되는 체크리스트 (순서 아님)



Persona / Role 역할·관점

어떤 전문가의 관점으로 일하게 할지 정한다

예) "보안을 중시하는 TypeScript 개발자로서"



Context 배경·제약

건드릴 파일 · 지켜야 할 규칙 · 참고 자료

예) @src/auth.ts 수정 · 비밀번호 평문 저장 금지



Task 할 일·검증

구체적으로 무엇을 하고, 성공을 어떻게 확인하는가

예) 로그인 구현 → 실패 시 에러 반환 → npm test 통과



Format 산출물 형태

결과를 어떤 형태로 받을지 정한다

예) 변경 요약과 diff로 보여줘

네 조각을 이어 붙이면 곧 3.2절의 '좋은 프롬프트' — 필요 없는 요소는 건너뛰어도 됩니다.

3.4 핵심 기법 — 결과를 바꾸는 6가지 도구

P·C·T·F에 없으면 결과가 확 달라지는, 검증된 프롬프트 기법 (필요한 것만 골라 쓰기)

1

역할 부여 Role · Persona

누구의 관점으로 일하게 할지 정한다

예) "보안을 중시하는 TypeScript 개발자로서"

2

예시 제공 Few-shot

원하는 형식·패턴을 예시로 보여준다

예) 입력→출력 한 쌍을 붙여 출력 형식을 고정

3

계획 먼저 Plan · 단계적 사고

바로 고치지 말고 계획부터 세우게 한다

예) "수정 전에 먼저 계획을 제안해줘 (read-only)"

4

작게 쪼개기 To-Do

큰 일을 한 항목씩 체크리스트로 나눈다

예) "이 To-Do를 한 항목씩, 한 커밋씩 진행"

5

검증 지시 Verification

끝났는지 아는 법을 함께 준다

예) "구현 후 npm test 통과까지 확인"

6

반복 개선 Iterate

한 번에 X — 대화로 다듬는다

예) 안 되면 증상을 구체화하거나 새 스레드로

3.5 논문으로 검증된 26가지 원칙 ①

Bsharat 외(MBZUAI, 2023) — GPT-4 기준 응답 품질 평균 57.7%↑ · 정확도 36.4%↑ (ATLAS 벤치마크, 원칙당 질문 20개)

① 프롬프트 구조와 명확성

#	원칙	예시
2	청중을 명시한다 ★	"독자는 이 분야 전문가다"
4	"하지 마" 대신 "하라"로 긍정 지시	"짧게 써"(○) / "길게 쓰지 마"(△)
5	쉬운 설명을 요청한다 ★	"11살에게 설명하듯", "입문자에게"
7	예시를 준다 (Few-shot)	입력 → 출력 예시 2~3개
8	###지시### ###예시### ###질문###로 구역을 나눈다	지시 · 예시 · 질문 · 자료를 줄바꿈으로 분리
12	"단계별로 생각해"를 쓴다	수학 · 논리 · 디버깅
17	구분자를 쓴다	"" ... "", <자료>...</자료>
20	답변의 첫머리를 프롬프트 끝에 써 준다 (출력 유도)	"...요약:" 으로 끝내기

② 구체성과 정보

#	원칙	예시
13	편향을 피하라고 명시한다	"고정관념에 기대지 말고 편향 없이 답해"
15	가르치고 시험 보게 한다 ★	"[주제]를 가르치고 끝에 테스트를 내줘. 답은 미리 주지 마"
21	상세한 글을 요청한다	"필요한 정보를 모두 넣어 [주제]에 대해 자세히 써줘"
24	주어진 첫 문장에 이어 쓰게 한다	"이 문장으로 시작해 흐름을 유지하며 완성해"
25	지켜야 할 요구사항을 명시한다 ★	키워드 · 규칙 · 힌트 · 형식
26	샘플과 같은 문체로 쓰게 한다 ★	"제공한 문단과 같은 언어 · 문체로 써줘"

★ 대형 모델에서 효과가 가장 컸던 원칙

2 · 5 · 14 · 15 · 16 · 25 · 26 — 특히 14번(모델이 먼저 질문)은 적용한 모든 질문에서 개선

출처 · Bsharat, S. M., Myrzakhan, A., & Shen, Z. (2023). Principled Instructions Are All You Need for Questioning LLaMA-1/2, GPT-3.5/4. MBZUAI, arXiv:2312.16171

3.5 논문으로 검증된 26가지 원칙 ②

사용자 상호작용 · 내용과 언어 스타일 · 복잡한 작업과 코딩 — ★ 효과가 특히 컸던 원칙 · △ 효과 불안정(3.6 참고)

③ 사용자 상호작용

#	원칙	예시
14	모델이 먼저 질문하게 한다 ★	“충분한 정보가 모일 때까지 나에게 질문해” — 효과 최상
11	자연스러운 사람 말투를 요청한다	“자연스럽고 사람처럼 답해”
22	문체는 유지하고 문법 · 어휘만 고치게 한다	“격식체는 격식체로 유지하며 문법만 교정”

⑤ 복잡한 작업과 코딩

#	원칙	예시
19	단계별 사고(CoT)와 예시(Few-shot)를 결합한다	추론 과정이 담긴 예시를 제공
23	여러 파일에 걸친 코드는 파일을 만들어 주는 스크립트로 받는다	“파일을 생성하는 [언어] 스크립트로 줘” — Codex는 파일을 직접 편집

④ 내용과 언어 스타일

#	원칙	예시
1	간결한 답을 원하면 인사말(“부탁해요·감사해요”)을 생략한다	바로 본론
3	복잡한 작업은 대화로 쪼개서 진행한다	조사 → 개요 → 초안 → 수정
6	“더 좋은 답에 팁 \$xxx를 주겠다”를 덧붙인다 △	효과 불안정 — 3.6 참고
9	“Your task is”, “You MUST”를 쓴다 △	“너의 임무는 ...이다”
10	“틀리면 불이익(penalized)을 받는다”를 쓴다 △	효과 불안정 — 3.6 참고
16	역할을 부여한다 ★	“너는 시니어 보안 리뷰어다”
18	핵심 단어 · 구절을 반복한다	중요한 조건을 두 번 언급

출처 · Bsharat, S. M., Myrzakhan, A., & Shen, Z. (2023). Principled Instructions Are All You Need for Questioning LLaMA-1/2, GPT-3.5/4. MBZUAI, arXiv:2312.16171

3.6 2026년 기준으로 본 26가지 원칙

논문은 2023년 말 GPT-3.5/4 · LLaMA 실험 결과입니다 — 최신 모델에서는 원칙마다 유효성이 다릅니다

✓ 지금도 확실히 유효한 원칙

2 청중 명시

3 작업 분해

7 · 19 예시와 CoT

8 · 17 구조화 · 구분자

14 모델이 먼저 질문

16 역할 부여

25 요구사항 명시

26 문체 샘플

→ 현재 OpenAI · Anthropic 공식 가이드와 일치

△ 효과가 약하거나 불안정한 원칙

6 · 10 팁 주기 · 불이익 경고

후속 연구에서 효과가 일관되지 않았습니다

9 “You MUST” 같은 강한 표현

최신 모델에서 과잉 반응 → 이유를 설명하는 차분한 지시를 권장

12 “단계별로 생각해”

추론 모델(GPT-5 · Claude 확장 사고)은 이미 내부에서 단계적으로 사고 → 효과 감소

1 인사말 생략

성과와 거의 무관 — 간결성에만 영향

Codex에서 쓰는 법 — 3.3 P · C · T · F에 원칙을 대응시키기

P 역할 · 관점

16 역할 부여

C 배경 · 제약

2 청중 · 25 요구사항 · 26 문체

T 할 일 · 검증

3 작업 분해 · 14 먼저 질문 · 19 CoT

F 산출물 형태

8 · 17 구조 · 20 출력 유도

출처 · Bsharat 외(2023) arXiv:2312.16171 · OpenAI 「Prompt engineering」 가이드(platform.openai.com) · Anthropic 「Prompt engineering overview」(docs.claude.com)

3.7 ▶ 실습 — 나쁜 한 줄을 좋은 프롬프트로

공통

P·C·T·F와 핵심 기법을 세 가지 상황에 직접 적용해 봅니다

1

① 계획부터

공통

PRD로 To-Do 뽑기

✗ “테트리스 만들어줘”

✓ @PRD.md 를 읽고 — 수정하지 말고(read-only) — Tetris MVP 완성을 위한 To-Do를 체크리스트로 먼저 만들어줘.

2

② 구현+검증

공통

한 기능을 검증까지

✗ “점수 기능 만들어줘”

✓ 꼼꼼한 프론트엔드 엔지니어로서 @script.js 에 줄이 삭제될 때 점수가 오르는 로직을 구현. 한 줄=100점, 여러 줄 동시 삭제 시 보너스. 완료 후 브라우저에서 점수가 정상적으로 오르는지 확인하는 방법을 알려줘.

3

③ 디버깅

공통

증상을 구체적으로

✗ “게임이 안 돼요, 고쳐줘”

✓ 블록이 좌우 벽을 통과해요. 벽 경계에서 멈추도록 충돌 검사를 고치고, 고친 뒤 브라우저에서 좌우 이동이 정상인지 확인하는 방법을 알려줘.

세 예시 모두 3.2절의 두 원칙과 3.3절의 P·C·T·F, 3.5~3.6절의 검증된 원칙이 그대로 적용됩니다 — 대상·규칙·검증을 빠짐없이.



PART 4

컨텍스트 엔지니어링

정의와 네 가지 전략 — 작성·선택·압축·격리

프롬프트가 '무엇을 시키나'라면, 컨텍스트는 '무엇을 알고 일하나'입니다. 필요한 정보를 '필요한 만큼만, 정확히' 보여 주는 기술입니다.

🕒 1교시 · 이론

4.1 컨텍스트의 정의

프롬프트가 '무엇을 시키나'라면, 컨텍스트는 '무엇을 보고 일하나' — 모델이 한 번에 보는 정보 전체

Anthropic · 기술적 정의

「Effective context engineering for AI agents」 2025.9

컨텍스트 = LLM이 응답을 생성할 때 포함되는 토큰의 집합

컨텍스트 엔지니어링 = 추론 중 최적의 토큰(정보) 집합을 선별 · 유지하는 전략의 총체

핵심: 결과 확률을 최대화하는 가장 작은 고신호 토큰 집합

안드레이 카파시 · 작업 메모리

X 게시물 2025.6

“다음 단계에 딱 맞는 정보로 컨텍스트 창을 채우는 섬세한 예술이자 과학”

비유: 모델 = CPU, 컨텍스트 창 = RAM

→ 제한된 메모리에 무엇을 올릴지 정하는 일

토비 뢰트케 · 필요한 맥락 전부

Shopify CEO · X 게시물 2025.6

“LLM이 과제를 그럴듯하게 풀 수 있도록 필요한 맥락을 모두 제공하는 기술”

카파시 발언의 계기가 된 표현

관점	컨텍스트를 무엇으로 보는가	Codex에서는 (4.2 네 가지 전략)
카파시	다음 단계에 딱 맞는 정보로 채워야 할 작업 메모리(컨텍스트 창)	@파일로 필요한 것만 — 선택
Anthropic	결과 확률을 최대화하는 최소한의 고신호 토큰 집합	컴팩션 · 스레드 분리 — 압축 · 격리
앤드류 응 팀 (Context Hub, 2026.3)	필요할 때 불러오고 계속 갱신되는 LLM 친화적 마크다운 파일	AGENTS.md · 스킴 — 작성

출처 · Anthropic 「Effective context engineering for AI agents」(2025.9.29) · A. Karpathy, X(2025.6.25) · T. Lütke, X(2025.6) · DeepLearning.AI 「Context Hub」(github.com/andrewyng/context-hub, 2026.3)

4.2 컨텍스트 엔지니어링 — 네 가지 전략

프롬프트가 '무엇을 시키나'라면, 컨텍스트는 '무엇을 알고 일하나'

핵심 비유 — LLM은 컴퓨터, 컨텍스트 윈도우는 작업 기억(RAM). '많이 넣을수록 똑똑'은 오해 — 정확히, 관련 있는 것만.

● 작성 (Write)

중요한 규칙·결정을 밖에 적어 두기

→ AGENTS.md에 저장 (Part 5)

● 선택 (Select)

지금 작업에 필요한 것만 골라 넣기

→ @로 파일 코크 집어 주기

● 압축 (Compress)

길어진 대화를 요약해 자리 확보

→ 컴팩션 · 스레드 나누기

● 격리 (Isolate)

작업별로 맥락을 분리하기

→ 한 작업 = 한 스레드



PART 5

Codex 핵심 기술

지시·권한 체계 · AGENTS.md · config.toml · Sandbox/Approvals

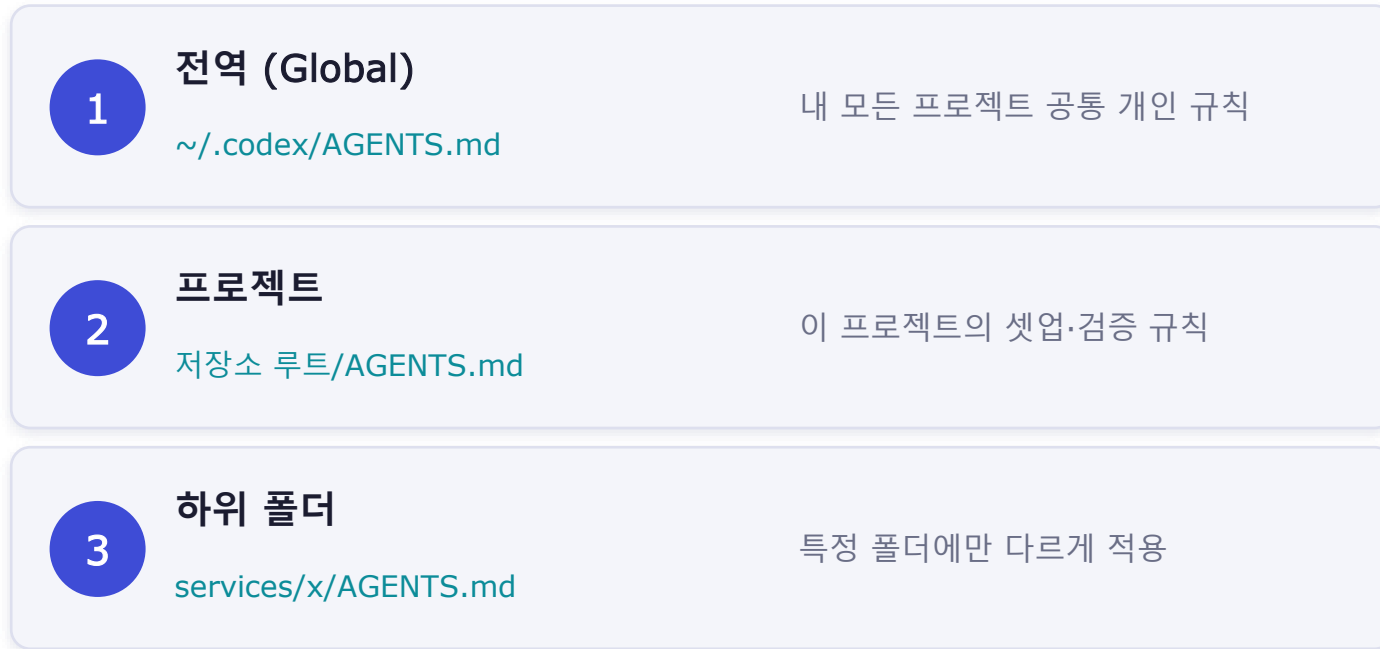
AGENTS.md는 상시 규칙, config.toml은 기본 동작 설정, Sandbox·Approvals는 권한·승인을 정합니다. 이 셋이 '하네스'의 토대가 됩니다.

🕒 1교시 · 이론

5.1 AGENTS.md — 작업 전에 먼저 읽는 규칙서

공통

프로젝트의 상시 규칙(컨텍스트) · 계층적으로 합쳐짐



우선순위: 전역 → 프로젝트 루트 → 현재 폴더 (가까울수록 우선)

이 파일이 곧 '컨텍스트'이자 '하네스'

'변경 시 반드시 lint·test를 통과' 같은 규칙을 적어 두면, 매번 말하지 않아도 Codex가 스스로 지킵니다.

▶ 실습

- 전역 AGENTS.md 만들기
- 프로젝트 AGENTS.md 만들기
- 로드되는지 확인 요청하기

config.toml — 모델·승인·샌드박스 기본값 저장 (~/.codex/config.toml)

5.2 Sandbox & Approvals — 안전하게 실행

샌드박스 = '무엇을 할 수 있나(범위)' · 승인 = '언제 물어보나(시점)'

읽기 전용

sandbox = read-only

수정 없이 분석·계획만. 가장 안전

가장 안전

Auto (권장)

workspace-write + on-request

작업 폴더 내 자동, 폴더 밖·네트워크는 승인

균형 · 권장

Full access

danger-full-access + never

제한 없음. 격리된 VM/CI에서만

위험 · 격리 환경

네트워크는 기본 차단(보안). 패키지 설치처럼 필요할 때만 '이번에만 허용'을 누릅니다. App은 승인 모드 버튼 한 번으로 전환.



PART 6

하네스(Harness) 엔지니어링

Agent = Model + Harness · 검증 · 루프

하네스는 Codex의 작업을 감싸는 '검증·안전 장치 묶음'입니다. 만든 결과가 '진짜 동작하는지' 자동으로 확인하게 만드는 틀입니다.

🕒 1교시 · 이론

6.1 하네스의 정의 — Agent = Model + Harness

모델이 지능을 제공한다면, 하네스는 그 지능을 실제로 일하는 시스템으로 바꿉니다

Agent
= **Model**
+ **Harness**

“하네스는 모델 자체가 아닌 모든 코드, 설정, 실행 로직이다.”

— 비벡 트리베디, LangChain (2026.3)

비유 모델 = 엔진 · 하네스 = 차체 · 핸들 · 브레이크

하네스에 들어가는 것 → Codex에서는

1 시스템 프롬프트 · 상시 규칙

AGENTS.md (5.1)

2 도구 · 스킬 · MCP와 그 설명

플러그인 · 스킬 · MCP (2.5)

3 파일시스템 · 샌드박스 · 브라우저

Sandbox · 인앱 브라우저 (5.2)

4 오케스트레이션 — 서브에이전트 · 모델 라우팅

스레드 · 모델 선택 (6.5)

5 결정적 처리를 하는 후 — 압축 · 린트 검사

빌드 · 테스트 · 컴팩션 (6.4)

출처 · Vivek Trivedy, 「The Anatomy of an Agent Harness」, LangChain Blog (2026.3.10)

6.2 하네스 엔지니어링 — 인물·기관별 정의

2025년 말 ~ 2026년 초에 정립된 용어 — 공통점은 '실수를 구조로 막는다'는 것

출처	정의	관점
미첼 하시모토 HashiCorp 공동창업자 · 블로그 「My AI Adoption Journey」(2026.2)	“에이전트가 실수하는 것을 발견할 때마다, 그 실수를 다시는 하지 않도록 해결책을 설계하는 데 시간을 쓰는 것.” 주로 AGENTS.md와 설정 파일을 고쳐서 실천합니다.	실수 → 구조적 재발 방지. 용어를 대중화
OpenAI · 라이언 로포폴로 「Harness engineering: leveraging Codex in an agent-first world」(2026.2)	“사람은 방향을 잡고, 에이전트가 실행한다.” (Humans steer. Agents execute.) 사람의 일은 코드 작성 → 시스템 설계 · 의도 명세 · 피드백 장치 구축으로 이동	5개월간 사람이 직접 짠 코드 없이 약 100만 줄 개발
비르기타 뵉켈러 Thoughtworks · martinowler.com (2026.4)	“AI 에이전트에서 모델을 제외한 모든 것. 결과물의 신뢰성을 높이기 위한 주변 인프라.”	제어 이론 관점 — 가이드 · 센서 (6.3)
Anthropic 「Effective harnesses for long-running agents」(2025.11)	여러 세션에 걸쳐 오래 일하는 에이전트가 진행 상황을 잃지 않게 하는 실행 구조 진행 기록 파일 · 기능 목록 · git 커밋 · 테스트 활용	긴 작업의 연속성

출처 · M. Hashimoto, 「My AI Adoption Journey」 mitchellh.com(2026.2) · R. Lopopolo, openai.com/index/harness-engineering(2026.2.11) · B. Böckeler, martinowler.com(2026.4.2) · Anthropic Engineering(2025.11)

6.3 가이드와 센서 — 세 엔지니어링의 관계

하네스는 행동 전에 방향을 잡는 '가이드'와 행동 뒤에 교정하게 하는 '센서'로 구성됩니다 (보켈러)

구분	가이드 (사전 제어 · Feedforward)	센서 (사후 제어 · Feedback)
역할	행동하기 전에 방향을 잡는다	행동한 뒤에 관찰하고 스스로 교정하게 한다
결정적 수단 (연산형)	규칙 파일 · 스케레톤 · 아키텍처 경계 → AGENTS.md · config.toml	테스트 · 린터 · 타입 검사 → npm test · npm run build
의미 기반 수단 (추론형)	스킬 · 역할 지시 → 플러그인 · P·C·T·F의 P	리뷰 에이전트 · LLM-as-judge → Codex 코드 리뷰

세 엔지니어링의 관계

수준	최적화 대상	한 줄 정의	이 과정
프롬프트 엔지니어링	한 번의 요청	지시문을 잘 쓰는 기술	Part 3
컨텍스트 엔지니어링	모델이 보는 것 전체	컨텍스트 창을 딱 맞는 정보로 채우는 기술 (카파시)	Part 4
하네스 엔지니어링	에이전트가 일하는 시스템 전체	모델 외 모든 것을 설계해 실수를 구조적으로 막는 기술 (하시모토)	Part 5 · 6

출처 · B. Böckeler, 「Harness engineering for coding agent users」, [martinfowler.com\(2026.4.2\)](#) · A. Karpathy, [X\(2025.6.25\)](#) · M. Hashimoto, [mitchellh.com\(2026.2\)](#)

6.4 Codex 실습의 하네스 — 4요소

공통

Codex가 만든 결과가 진짜 동작하는지 자동으로 확인하게 만드는 안전망

1

빌드 (Build)

코드가 실제로 컴파일/번들되는
가

`npm run build / typecheck`

2

테스트 (Test)

기능이 의도대로 동작하는가

`npm test (Vitest)`

3

배포 (Deploy)

산출물을 실제로 배포할 수 있는
가

GitHub Pages

4

로그 (Log)

실행 결과·에러를 확인할 수 있는
가

테스트/런타임 로그

하네스 = '검증 가능한 규칙' + '권한 경계'. 일부러 코드를 깨보고(회귀 테스트), 빨간불이 났다가 수정 후 다시 통과하는 흐름을 직접 보면 '안전망'의 감이 옵니다.

6.5 루프 엔지니어링과 모델 엔지니어링

하네스 다음 단계 — 반복을 자동화하고(루프), 단계마다 모델을 배분합니다(모델)

용어	정의	출처
루프 엔지니어링 Loop Engineering	사람이 틴마다 지시하지 않아도 에이전트를 반복 실행하고, 결과를 검증하고, 스스로 이어가게 하는 자동화된 반복 구조의 설계	애디 오스마니 (2026.6) 계기: 보리스 체르니 "이제 Claude에게 프롬프트를 쓰지 않는다. 프롬프트를 쓰는 루프를 쓴다."
앤드류 응의 3중 루프	① 에이전트 코딩 루프 (작성 → 테스트 → 반복) ② 개발자 피드백 루프 (사람의 검토 · 방향 조정) ③ 외부 피드백 루프 (사용자 반응)	ADTmag (2026.7)
모델 엔지니어링 해석	단계마다 어떤 모델을 어떤 추론 강도로, 어떤 속도 · 비용으로 쓸지 설계하는 일	정립된 용어는 아님 Agent = Model + Harness의 'Model' 쪽 설계

⚠ 하네스가 루프의 전제 조건

테스트 · 규칙 · 샌드박스 없이 루프를 돌리면 잘못된 코드가 그만큼 빠르게 쌓일 뿐입니다. 6.1~6.4의 하네스를 먼저 갖춘 뒤 반복을 자동화합니다.

Codex App에서는

예약(Scheduled) 루프의 트리거 · 주기 (2.5)
스레드 분리 · 검증 지시 반복마다 격리 · 검증
모델 · 추론 강도 선택 모델 엔지니어링

출처 · codecentric 「Loop, Harness, Context Engineering: The Terms Explained」(2026) · ADTmag 「Loop Engineering Emerges as Developers Put AI Coding Agents on Repeat」(2026.7.1)

여기까지가 공통 개념입니다

이제 실습을 시작합니다

앞에서 배운 원리(프롬프트·컨텍스트·하네스)를 그대로 Tetris 만들기에 적용합니다.

실습 프로젝트

Tetris — 브라우저 게임

Part 7-8 · 프롬프트 복붙 · GitHub Pages 배포

실습 프로젝트 한눈에

프롬프트 복붙으로 만드는 브라우저 게임 — 이렇게 진행합니다

구분	공통
만드는 제품	Tetris · 브라우저 게임
코딩 사전지식	불필요 (프롬프트 복붙)
기술 스택	순수 HTML/CSS/JS
외부 연동	없음
배포 방식	GitHub Pages
모니터링	(선택) 없음
워크북 범위	Part 7-8

테트리스 만들기 로드맵

Part 7 → 8 · 프로그래밍 경험 0에서도 프롬프트 복붙만으로 완주



각 '프롬프트' 상자의 내용을 그대로 복사해 Codex 작성 칸에 붙여넣기만 하면 됩니다.

공통 · Part 7 — Tetris PRD → To-Do

공통 트랙

게임도 '무엇을 만들지' 글로 먼저 정의합니다

① PRD.md 작성

아래 내용을 그대로 붙여넣어 Tetris/PRD.md 로 저장하게 시킵니다.

```
# PRD — Tetris
```

```
## 목표
```

```
브라우저에서 도는 테트리스
```

```
## 핵심 기능
```

```
블록 낙하 · 회전 · 이동  
줄 삭제 · 점수 · 게임오버
```

② To-Do 도출 (읽기 전용)

@PRD.md 이 PRD를 읽고 Tetris MVP를 완성하기 위한 To-Do 목록을 만들어줘. 수정은 하지 마.

[TIP] 계획 단계는 read-only로 받으면 파일이 안 바뀌어 안전합니다.

PRD → To-Do 흐름은 어디서나 똑같습니다

PRD를 쓰고 → @로 참조해 To-Do를 받는 방법은 모든 프로젝트에 그대로 적용됩니다.

공통 · Part 8.1-8.5 — 게임 만들기

공통 트랙

프롬프트 0 → 7 을 순서대로 복사·붙여넣기

0

프로젝트 골격

순수 HTML/CSS/JS로 빈 보드·점수 영역

2

자동 낙하 & 충돌

0.5초마다 한 칸 내려오고 바닥에서 고정

4

라인 클리어 & 점수

가득 찬 줄 삭제 → 위 블록 내리고 점수 증가

6

다듬기 (선택)

다음 블록 미리보기·레벨·속도 증가

1

보드 & 블록

그리드와 테트로미노(블록 모양) 정의·렌더링

3

키보드 조작

←→ 이동, ↓ 빨리, ↑ 회전, Space 즉시 낙하

5

게임 오버 & 재시작

새 블록 공간 없으면 종료·다시 시작

7

테스트 점검

시나리오 점검하고 안 되는 항목 고치기

[TIP] 문제가 생기면 증상을 구체적으로 적어 디버깅 프롬프트로 (예: "블록이 벽을 통과해요").

공통 · Part 8.6-8.7 — 배포하기

공통 트랙

만든 게임을 누구나 접속하는 링크로 무료 공개

1

Git으로 기록

코드가 동작하면 버전을 기록(commit). VS Code '소스 제어' 패널에서 버튼으로도 가능 — 세 단어만: add → commit → push

2

GitHub에 올리기

GitHub CLI(gh)로 인증·저장소 생성이 쉬움. 또는 웹에서 빈 저장소를 먼저 만들고 push

3

GitHub Pages 배포

저장소 Settings → Pages 에서 공개. 정적 파일(HTML/CSS/JS)이라 무료 배포 가능 → github.io 링크 생성

✅ 완료 조건 블록 낙하·조작·줄 삭제·점수·게임오버가 모두 동작하고, GitHub Pages 주소로 접속된다.

Part 9 — 마무리 회고 & 부록

공통

이 자료로 개념을 정리하고 복습합니다

마무리 회고 (Part 9)

- 전체 흐름 한눈에 보기
- 핵심 개념을 '한 문장으로' 말해 보기
- 최종 점검 체크리스트
- 자주 막히는 문제 트러블슈팅

부록 — 언제든지 꺼내 쓰는 참고

- A 프롬프트·컨텍스트·하네스 치트시트
- B config.toml 전체 예시(주석 포함)
- C AGENTS.md 템플릿(복사용)
- D 용어집

한 문장으로 말해 보기 — 개념 회고

프롬프트

작게 쪼개고 검증 방법을 함께 준다

컨텍스트

필요한 것만, 정확히 (많이 ≠ 똑똑)

AGENTS.md

작업 전 먼저 읽는 상시 규칙서

하네스

결과가 진짜 동작하는지 검증하는 안
전망

핵심 한 장 요약

이것만 기억하면 됩니다

Codex는 에이전트다

챗봇이 아니라 파일을 직접 고치고 테스트까지 돌린다

프롬프트

작게 쪼개고, 검증 방법을 함께 준다

컨텍스트

많이가 아니라 '정확히, 필요한 것만'

규칙 & 안전

AGENTS.md에 규칙, Sandbox/Approvals로 안전

하네스

빌드·테스트·배포·로그로 결과를 검증

실습은 직접

Tetris — 프롬프트 복붙으로 브라우저 게임을 만들어 배포



수고하셨습니다

개념은 함께, 실습은 각자 — 이제 내 손으로 만든 결과물이 인터넷에 있습니다.

다음 단계

- 부록 A 치트시트로 P·C·T·F를 내 프로젝트에 적용해 보기
- AGENTS.md에 나만의 '완료의 정의' 못 박기
- 게임에 다음 블록 미리보기 · 레벨 · 속도 증가 기능 더해 보기

Q & A · 질문을 환영합니다